

Laboratorio 6 - Parte 3: BluePrints API - React - UI

mediante Autenticación JWT y Redux Toolkit

María Paula Rodríguez Muñoz

Juan Andrés Suárez

Juan Pablo Nieto

Tomás Felipe Ramírez

Materia: ARSW

20 de marzo de 2026

1. Introducción

Este laboratorio extiende un proyecto previo para la gestión de blueprints mediante la creación de un cliente moderno en ****React + Vite****. Su objetivo es implementar una interfaz de usuario reactiva que gestione y visualice planos geométricos, consumiendo una REST API protegida con autenticación JWT y criptografía RSA. La aplicación sigue una arquitectura por capas en el frontend e integra procesamiento visual en tiempo real mediante un lienzo (Canvas). Este reporte detalla la implementación técnica, los objetivos alcanzados y las evidencias de funcionamiento del sistema final.

2. Objetivos

2.1. Objetivo General

Desarrollar un cliente moderno en React para la gestión de blueprints que implemente autenticación segura con JWT, manejo de estado global con Redux Toolkit y una arquitectura desacoplada por capas para la integración con APIs REST, garantizando la mantenibilidad y escalabilidad del sistema.

2.2. Objetivos Específicos

- Implementar una interfaz de usuario mediante componentes de React que facilite la visualización interactiva de planos en el navegador.
- Integrar Redux Toolkit para la gestión centralizada de la información de autores, planos y el plano seleccionado actualmente.
- Desarrollar una capa de servicios (Service Layer) con un patrón de fachada que permita conmutar entre almacenamiento Mock y producción sin alterar la lógica de los componentes.
- Configurar autenticación de usuarios mediante tokens JWT almacenados de forma persistente en `localStorage` para asegurar el acceso a recursos protegidos.
- Implementar rutas protegidas mediante componentes de orden superior y manejo de estados asincronos para una experiencia de usuario fluida.
- Validar el funcionamiento del cliente mediante una suite de pruebas unitarias y de integración con Vitest.

2.3. Diagrama de Componentes

El sistema se organiza pedagógicamente en los siguientes módulos:

- **UI Components:** Pages (Blueprints, Login) y componentes especializados como `BlueprintCanvas`.
- **State Manager:** Redux Store con un slice dedicado a la lógica de negocio de los blueprints.

- **Service Layer:** Capa de abstracción encargada de la comunicación con el origen de datos (Mock/API).

3. Ejecución del Proyecto

3.1. Requisitos Previos

- Node.js 18+ instalado en el sistema operativo.
- Gestor de paquetes npm/npx configurado.
- Docker Desktop activo para el despliegue de contenedores.

3.2. Configuración de Entorno

La aplicación utiliza variables de entorno inyectadas mediante un archivo `.env` para configurar la URL base de la API:

```
1 VITE_API_BASE_URL=http://localhost:8080/api
2 VITE_USE MOCK=true
```

3.3. Compilación y Ejecución

Para iniciar el proyecto en modo de desarrollo, se deben ejecutar los siguientes comandos desde la terminal:

```
1 # Instalacion de dependencias del proyecto
2 npm install
3 # Levantar el servidor de desarrollo Vite
4 npm run dev
```

La aplicación estará accesible en el navegador a través de la dirección `http://localhost:5173`.

3.4. Endpoints Principales

Todas las operaciones sobre la ruta `/api/**` requieren un token JWT válido adjunto en las cabeceras de autorización.

3.4.1. Servicio de Autenticación

```
1 POST /auth/login
2 Content-Type: application/json
3 {
4   "username": "student",
5   "password": "student123"
6 }
```

3.4.2. Consulta de Blueprints de un Autor

```
1 GET /api/v1/blueprints/{author}
2 Authorization: Bearer <access_token>
```

3.4.3. Creación de un Nuevo Blueprint

```
1 POST /api/v1/blueprints
2 {
3   "author": "john",
4   "name": "workshop",
5   "points": [{"x": 10, "y": 10}, {"x": 50, "y": 50}]
6 }
7 # Código de respuesta esperado: 201 Created
```

3.5. Ejecución de Suite de Pruebas

Se validan los componentes y la lógica de estado mediante pruebas automatizadas:

```
1 # Ejecucion de pruebas unitarias
2 npm test
```

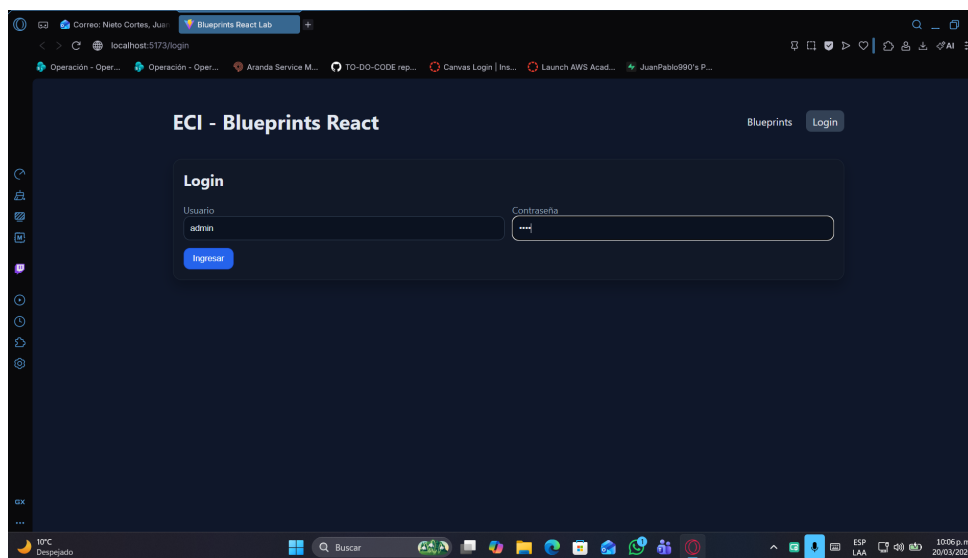


Figura 1: Figura 1: Resumen de ejecución de pruebas unitarias con Vitest

4. Evidencias de Ejecución

4.1. Proceso de autenticación JWT

Para acceder a los recursos protegidos se debe ingresar al sistema mediante la pantalla de login, la cual retorna un token JWT firmado.

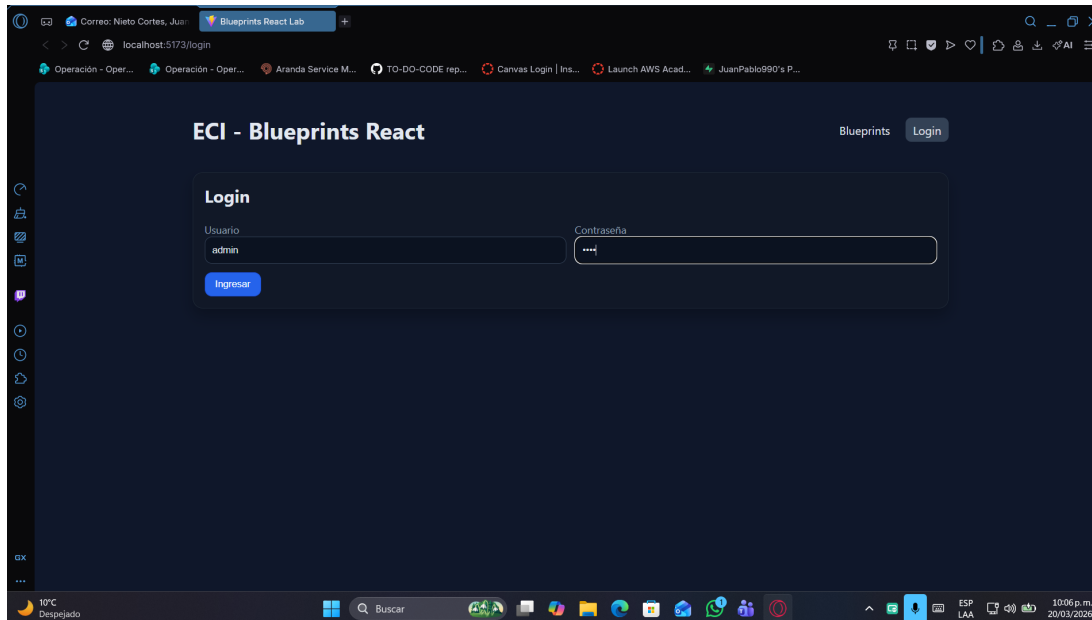


Figura 2: Figura 2: Solicitud de autenticación al endpoint `/auth/login`

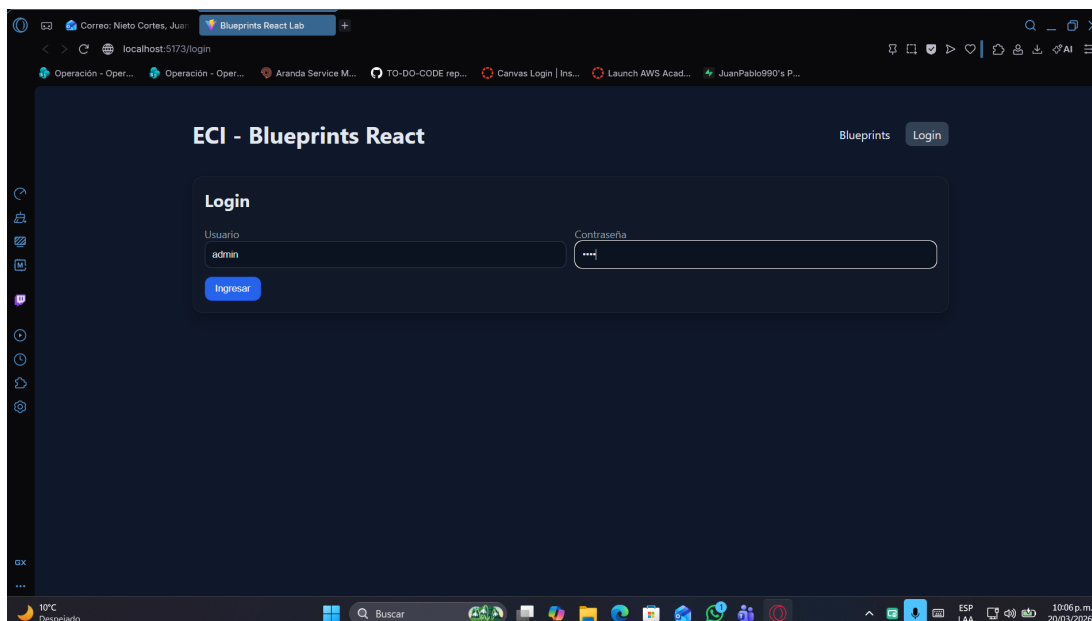


Figura 3: Figura 3: Token JWT capturado exitosamente por el frontend

4.2. Consulta por Autor y Cálculo de Puntos

Se verificó la funcionalidad de búsqueda por autor, la cual muestra la tabla de planos y calcula dinámicamente el total de puntos geométricos.

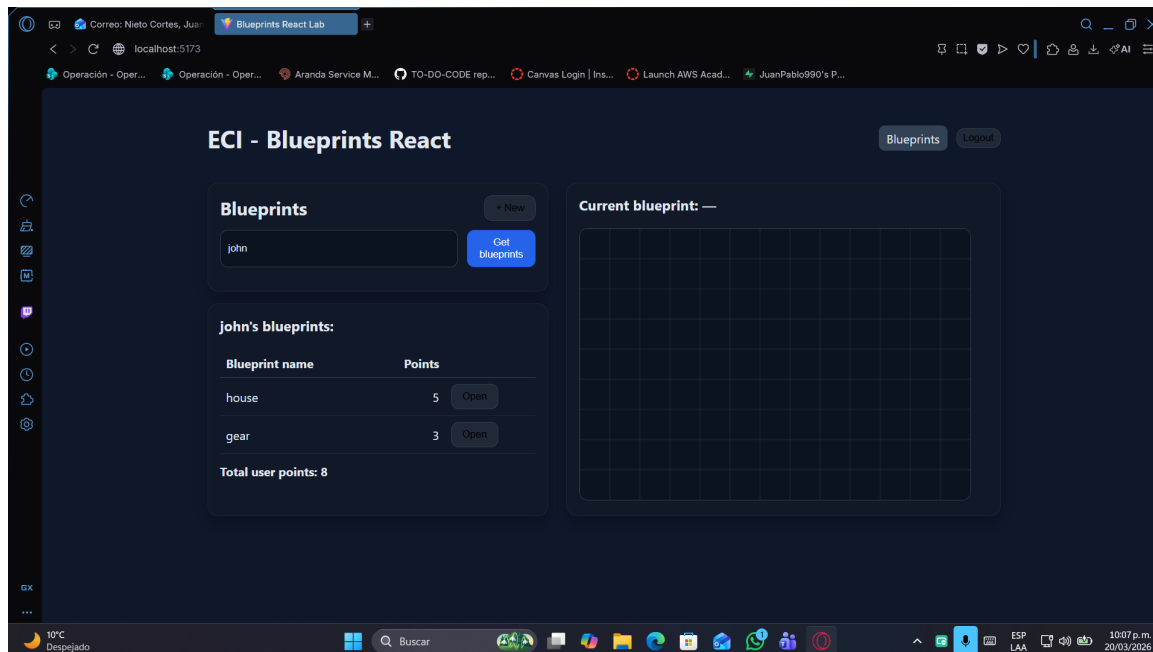


Figura 4: Figura 4: Interfaz de búsqueda y visualización de planos del autor john

4.3. Visualización en Canvas Interactiva

Al abrir un plano, se grafican los puntos y segmentos correspondientes en el lienzo centralizado.

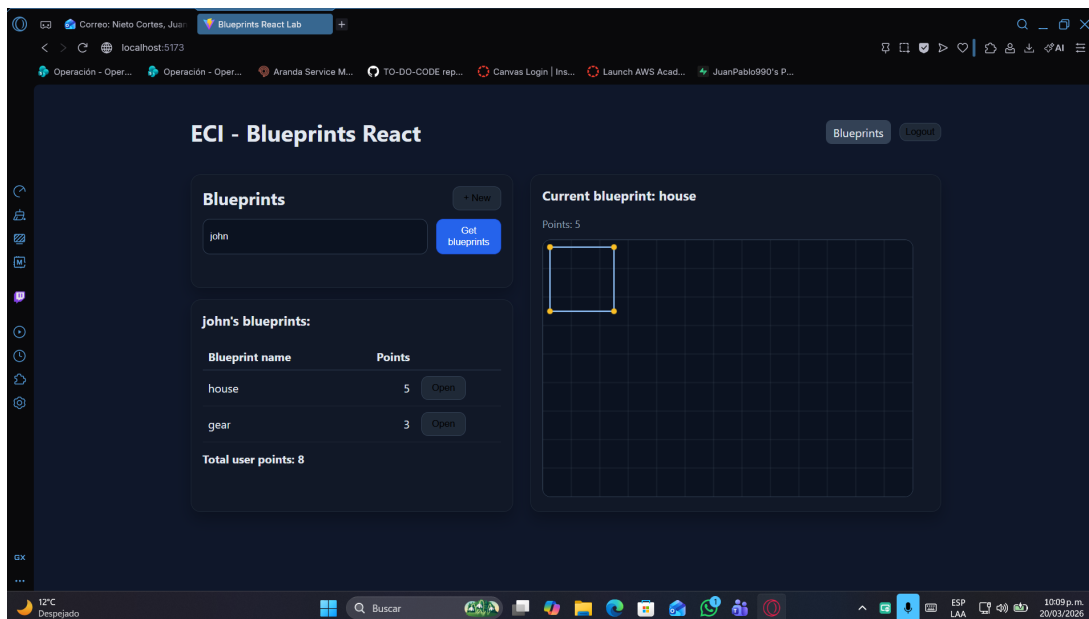


Figura 5: Visualización de la figura geométrica procesada en el Canvas

4.4. Persistencia: Creación de un Nuevo Plano

Se validó la capacidad de enviar nuevos blueprints al sistema mediante el formulario integrado.

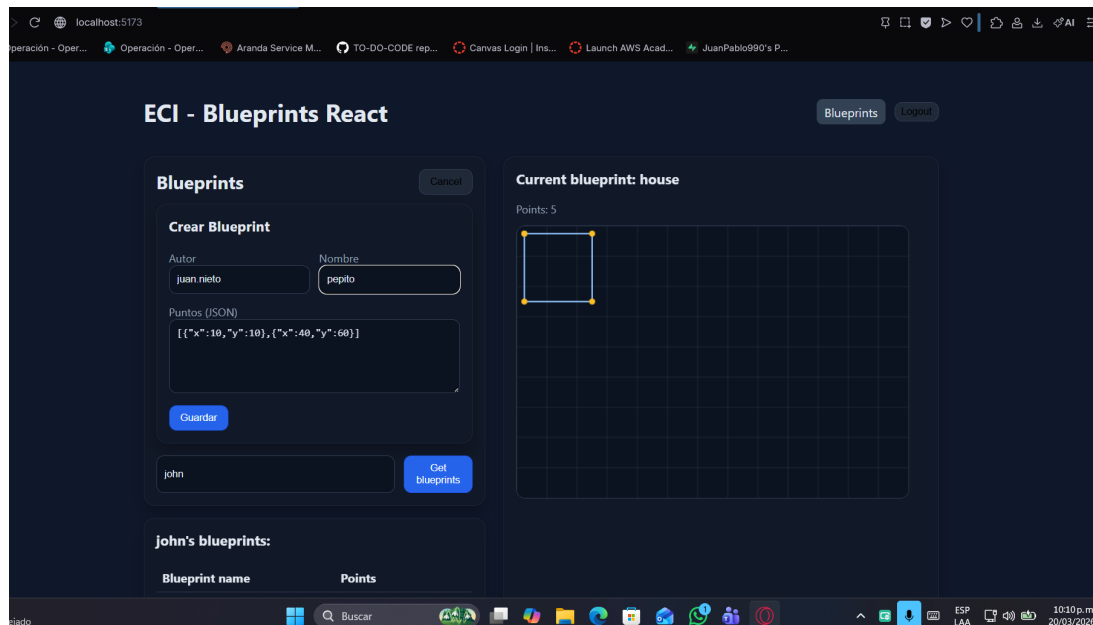


Figura 6: Figura 6: Solicitud POST para la creación exitosa de un blueprint

5. Conclusiones

Basado en el desarrollo realizado para este laboratorio, se destacan las siguientes conclusiones técnicas:

- Se logró implementar un cliente SPA seguro y funcional utilizando tecnologías de React y Redux Toolkit, cumpliendo con la arquitectura moderna de aplicaciones web.
- La integración de seguridad mediante JWT y el uso de interceptores RSA permitió establecer un mecanismo de autenticación robusto, escalable y acorde a las necesidades del proyecto.
- La arquitectura por capas implementada en el frontend, separando componentes, servicios y lógica de estado, facilitó significativamente la organización y mantenibilidad del código.
- El diseño de una capa de persistencia abstracta (Fachada de Servicios) permitió soportar almacenamiento Mock y producción sin acoplar los componentes visuales a la API.
- El uso de herramientas de calidad como Vitest, Docker y ESLint permitió desarrollar un producto final alineado con las mejores prácticas de la industria de software.

6. Resumen de la Entrega

Este proyecto representa una solución integral para el Laboratorio 6, integrando todas las competencias adquiridas en el curso ARSW respecto al desarrollo de frontends modernos y seguros. La aplicación final no solo cumple con los requisitos funcionales mínimos, sino que ofrece una experiencia de usuario superior mediante el manejo riguroso de estados y errores.